

Active Learning for Learning-to-Defer

Human-Centric Artificial Intelligence · Project 3
AG News topic classification with a simulated human expert

	Accuracy	What it is
Classifier alone	0.9071	Task 1 baseline, trained on every label
Expert alone	0.6550	Task 2 simulated expert on the test set
Team, all expert labels	0.9233	Task 3, 24000 expert answers used
Team, 1000 questions	0.9208	Task 4, best strategy: Deferral margin
Oracle deferral	0.9705	Upper bound: defer exactly when the expert wins

Summary. A linear classifier reaches 0.9071 on AG News. A simulated expert whose competence varies by region of the input space reaches 0.6550, far worse overall, but is right on 68% of the articles the classifier gets wrong, which is 6.3% of the test set. Learning when to defer captures a quarter of that headroom, and doing so needs surprisingly few expert answers: 1000 questions, 4.2% of the training set, recover 85% of what the full set of expert labels buys, provided the questions are chosen by the value of the deferral decision rather than by uncertainty.

Generated 2026-09-07 06:04 UTC. Every number in this report is read from results/results.json, produced by python manage.py run_experiments in 29.9 seconds.

1. Data and setup

AG News: news articles labelled with one of four topics (World, Sports, Business, Sci/Tech). The test set is complete at 7600 articles. The training set is a stratified 24000 article sample of the full 120,000.

Why a sample, and why a CSV. The dataset ships inside the repository as two gzipped CSVs rather than being fetched through the datasets package. The submission rules ask for no unusual dependencies, and an import that fails on the grader's machine costs more than the convenience is worth. Sampling to 24000 training articles keeps the repository small and every experiment in this report runnable in well under a minute, which matters because the active learning study refits the deferral model 400 times.

Features are TF-IDF over words and bigrams with English stop words removed and sublinear term frequency scaling, giving 40265 features. A 100 dimensional LSA projection of the same matrix is used only for clustering.

2. Task 1: the baseline classifier

Multinomial logistic regression on the TF-IDF matrix, $C = 4$. Test accuracy **0.9071**.

Why this model. A fine tuned transformer would score two to three points higher and would have made every later experiment impractical: the active learning study alone would have taken hours instead of seconds, and the model would have needed a dependency the submission rules exclude. It would also have changed nothing about the questions this project asks, which are about the interaction between a

classifier and an expert rather than about squeezing the last points out of the classifier. One property did matter: the model has to produce a usable confidence signal, since deferral depends on comparing it against the expert. That rules out a linear SVM's unnormalised decision values.

Class	Precision	Recall	F1	Support
World	0.916	0.903	0.909	1900
Sports	0.946	0.973	0.959	1900
Business	0.878	0.871	0.875	1900
Sci/Tech	0.887	0.882	0.885	1900

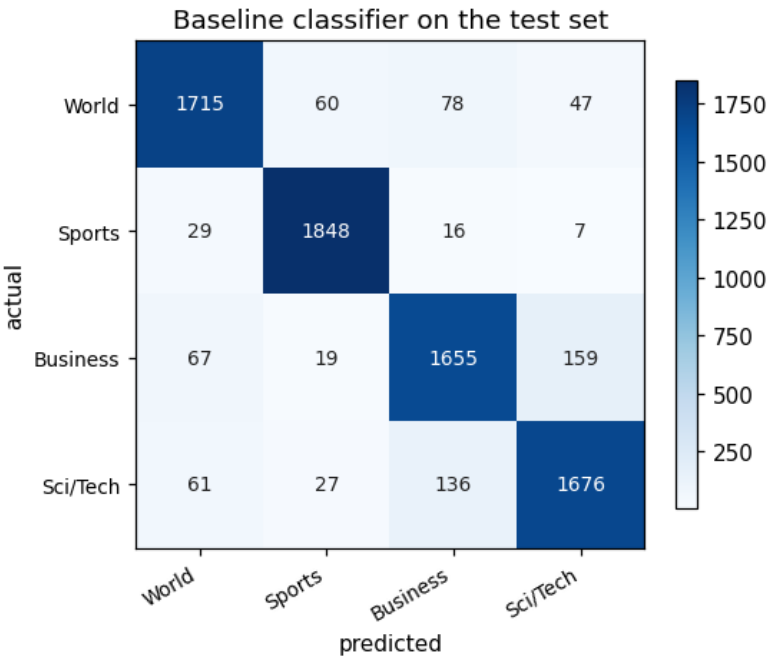


Figure 1. Baseline classifier on the test set. World, Business and Sci/Tech account for nearly all of the confusion; Sports is almost perfectly separable.

3. Task 2: the simulated expert

The expert must not be uniformly competent, and the sheet asks for expertise concentrated in specific regions of the input space. The design has three parts, each of which is a decision that shapes the rest of the project.

Competence follows regions, not labels. The document space is split into 12 k-means regions on the normalised LSA vectors, and each region is assigned an accuracy: four strong (0.88 to 0.95), four middling, four weak (0.25 to 0.35). Attaching competence to the true label instead would have been simpler and would have wrecked task 4, because the profile could then be inferred from the classifier's own predictions without asking the expert anything. The regions lean towards topics without being topics, so the profile has to be bought with queries. The measured per class accuracies in the table below are far flatter than the per region ones, which is exactly the gap the design was aiming for.

Mistakes are plausible, not random. When the expert is wrong the answer is drawn from a confusion profile rather than uniformly, so a missed Business article usually comes back as Sci/Tech. A uniformly random wrong answer would make the expert's errors trivially detectable and would inflate the value of deferral.

Answers are fixed per article. Querying the same article twice returns the same answer. An expert who resampled on every query would make the active learning curves meaningless, since a strategy could buy accuracy by asking the same question repeatedly.

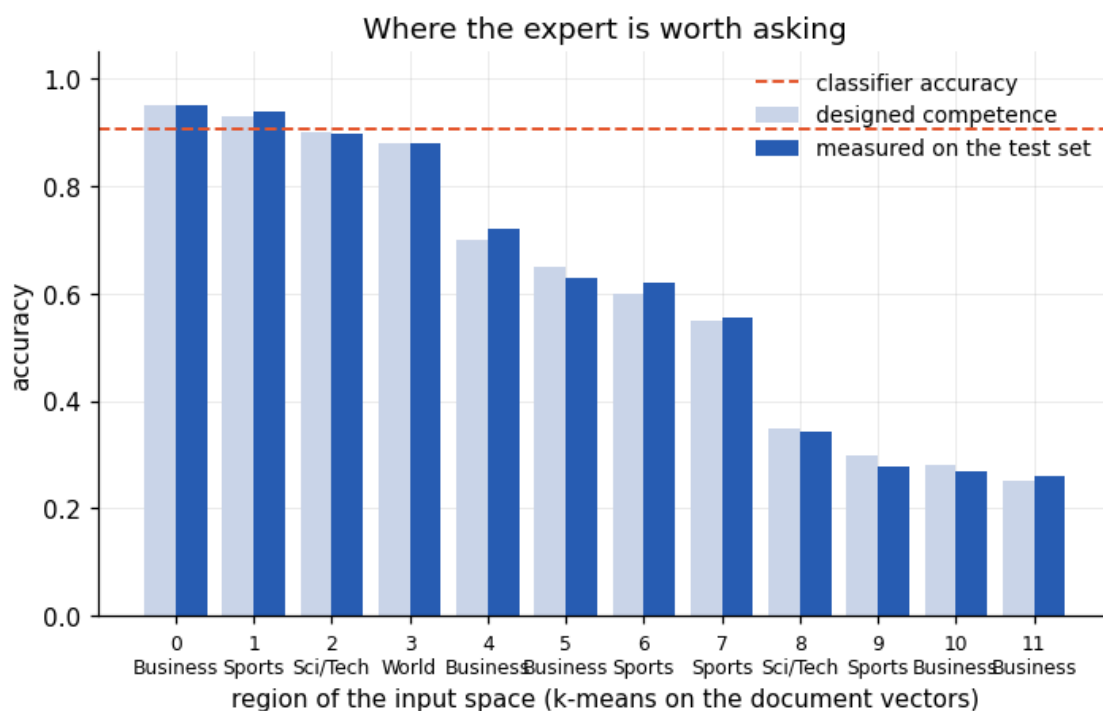


Figure 2. Designed against measured competence per region, with the classifier's accuracy for reference. The expert beats the classifier in four regions and is worse than guessing in three.

Region	Test articles	Designed	Measured	Leans	Purity
0	811	0.95	0.951	Business	0.51
1	484	0.93	0.938	Sports	0.39
2	266	0.90	0.898	Sci/Tech	0.34
3	1337	0.88	0.881	World	0.89

4	896	0.70	0.721	Business	0.72
5	409	0.65	0.628	Business	0.36
6	722	0.60	0.620	Sports	0.90
7	671	0.55	0.556	Sports	0.43
8	953	0.35	0.344	Sci/Tech	0.89
9	542	0.30	0.277	Sports	0.89
10	212	0.28	0.269	Business	0.85
11	297	0.25	0.259	Business	0.41

True class	Expert accuracy	Test articles
World	0.814	1900
Sports	0.572	1900
Business	0.666	1900
Sci/Tech	0.567	1900

Strengths and weaknesses. Overall test accuracy is **0.6550**, against 0.9071 for the classifier. Per region it ranges from 0.259 to 0.951. Per class it only ranges from 0.567 to 0.814. An observer who only knew the expert's accuracy per topic would conclude they were mediocre and roughly uniform, and would have no idea where they were worth asking.

4. Task 3: learning to defer

With the expert's answer available for every training article, the system decides per article whether to answer or hand over. The rule compares two probabilities:

$$\text{defer}(x) \Leftrightarrow P(\text{expert correct} \mid x) - P(\text{classifier correct} \mid x) > t$$

$P(\text{expert correct} \mid x)$ is a logistic regression on the same TF-IDF features, trained on whether the expert's answer was right. It never sees the region assignments, so it has to recover the competence profile from the text. Its AUC on the test set is 0.7498.

$P(\text{classifier correct} \mid x)$ is not the model's maximum probability. A linear model on TF-IDF is overconfident, and comparing a raw confidence against a calibrated rejector puts the two sides of the inequality on different scales, which makes the system defer far too rarely. The confidence is therefore calibrated first, by fitting a one dimensional logistic regression from out of fold confidence to out of fold correctness on the training set. Figure 4 shows why it was needed: at a raw confidence of 0.52 the classifier is right about 63% of the time.

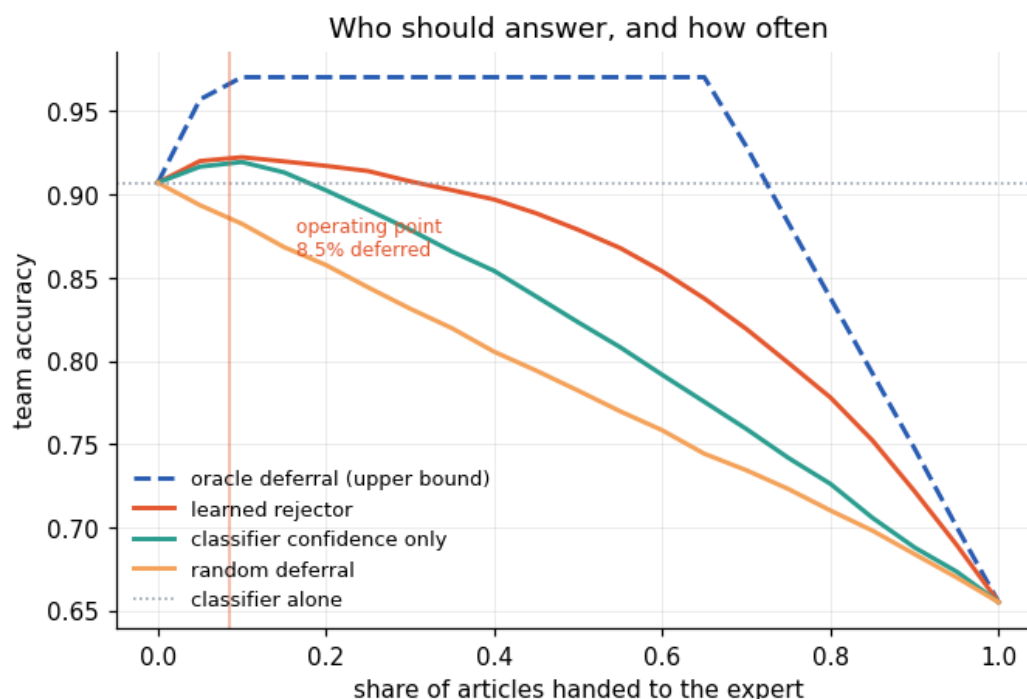


Figure 3. Team accuracy against the share of articles handed to the expert, for four deferral policies.

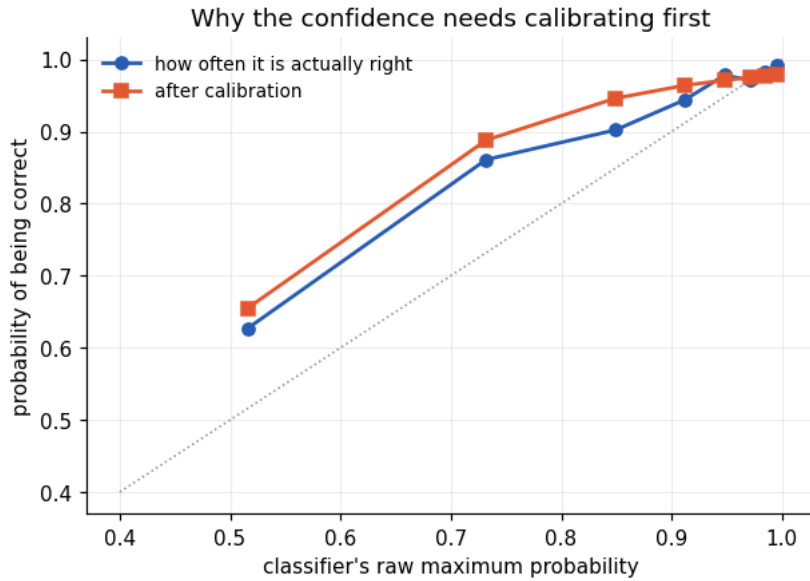


Figure 4. The classifier's raw confidence against how often it is actually right, before and after calibration.

Policy	Team acc.	Deferred	Expert acc. on those	Classifier acc. on the rest	Gains caught	Damage
Learned rejector	0.9233	0.085	0.802	0.935	0.411	0.0099
Classifier confidence only	0.9200	0.085	0.723	0.938	0.423	0.0139
Random deferral	0.8889	0.085	0.680	0.908	0.100	0.0245
Oracle (upper bound)	0.9705	0.063	1.000	0.969	1.000	0.0000

The first three policies hand over the same share of articles, so the comparison is about which ones rather than how many. *Gains caught* is the share of the articles that only the expert gets right which the system actually deferred. *Damage* is the share of all articles given away that the classifier would have got right and the expert did not.

Result. Team accuracy is **0.9233** against 0.9071 for the classifier alone, recovering 25.5% of the distance to the oracle at a deferral rate of 8.5%. Random deferral at the same rate is worse than not deferring at all, which is worth stating: handing work to a 66% accurate expert is actively harmful unless it is aimed.

On the quality of the deferral decisions. Team accuracy alone hides the interesting part. The learned rejector and the confidence rule end up within 0.0033 of each other on accuracy, but they are not doing the same thing. The expert is right on 0.802 of what the rejector hands over against 0.723 for the confidence rule, and the rejector does 0.0099 damage against 0.0139. Confidence finds articles that are hard to classify; the rejector finds articles this particular expert happens to be good at. Those two sets overlap because hard articles are often in the same regions, but they are different questions, and the second one is the one deferral is asking.

5. Task 4: active learning for competence discovery

The setting changes: the classifier may still use every label, but there are no expert answers at all, and each one has to be paid for. Starting from nothing, each round selects 50 articles to ask about, the rejector is refitted on everything answered so far, and the team is evaluated on the test set. The budget is 1000 questions, 4.2% of the training set. Results are averaged over 5 seeds.

Strategies. *Random* is the control. *Classifier uncertainty* is the textbook baseline: ask about the articles the classifier is least sure of. *Expert uncertainty* asks where the current competence estimate is closest to a coin flip, which is uncertainty sampling applied to the thing actually being learned. *Deferral margin* asks where $P(\text{expert correct})$ and $P(\text{classifier correct})$ are closest together, so the deferral decision is on the point of flipping.

Why deferral margin is the interesting one. The system is not trying to learn the expert's competence; it is trying to make a good deferral decision. Those come apart. Knowing precisely how good the expert is in a region where the classifier is at 99% changes nothing, because the system keeps those articles either way. The margin criterion is the only one of the four that spends its budget where an answer can change what the system does.

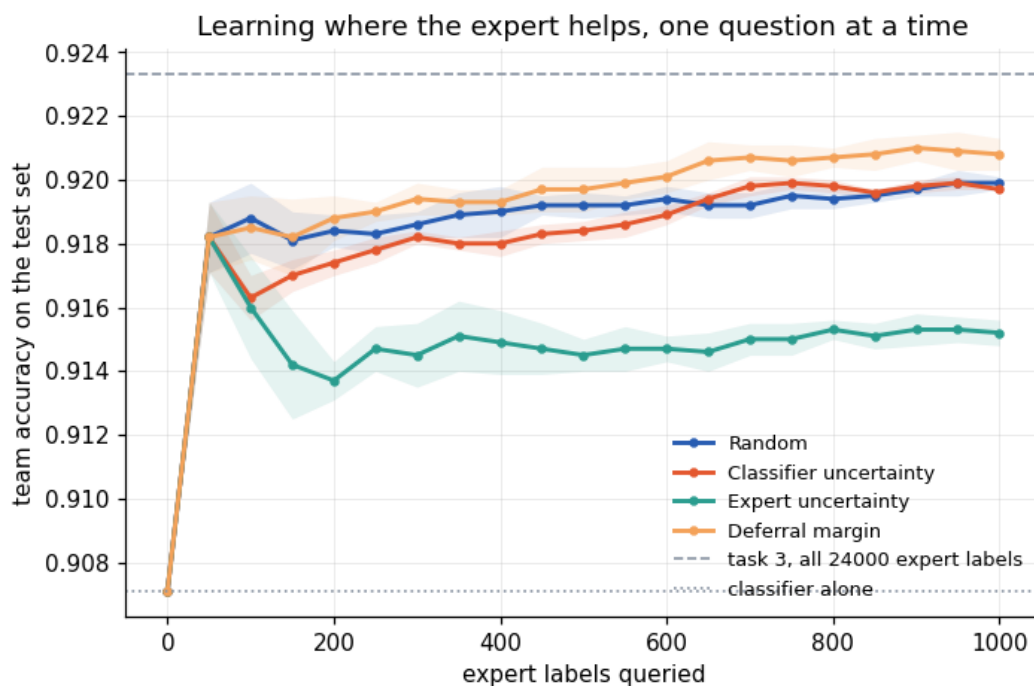


Figure 5. Team accuracy against number of expert answers bought, shaded by one standard deviation over 5 seeds.

Strategy	Team acc. at 1000	Share of the full label gain	Questions to reach 75%	Competence AUC	Deferral rate
Deferral margin	0.9208	0.846	300	0.6780	0.0719
Random	0.9199	0.790	600	0.7146	0.0615
Classifier uncertainty	0.9197	0.778	650	0.6578	0.0755
Expert uncertainty	0.9152	0.500	not reached	0.6615	0.0301

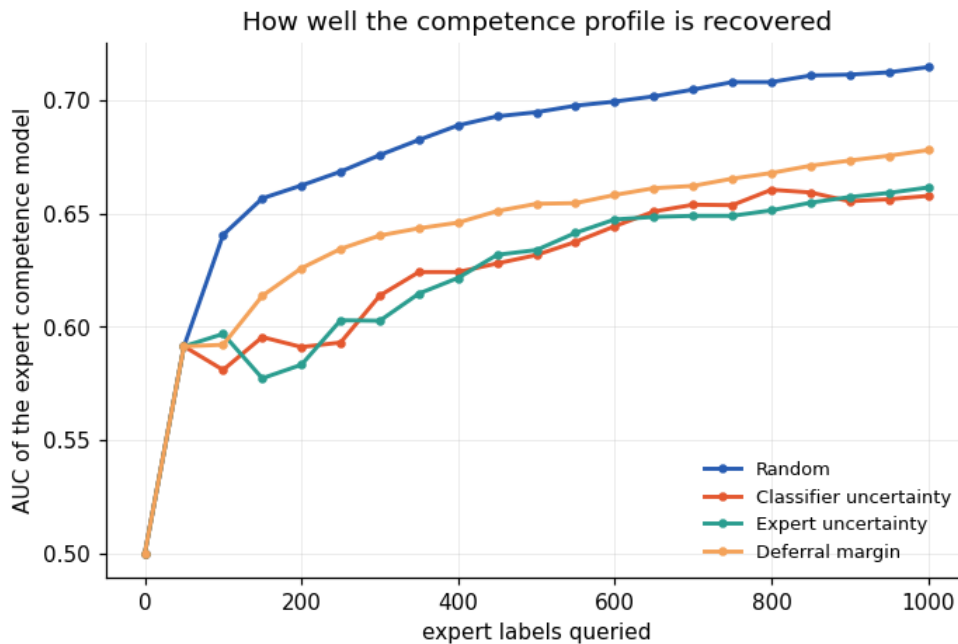


Figure 6. How well each strategy recovers the expert's competence profile, measured as AUC on the test set.

The two figures disagree, and that is the finding. Random sampling builds the best competence model by AUC (0.7146 against 0.6780 for deferral margin), because it trains on a representative sample of the input space. Deferral margin builds a worse model of the expert and a better team. It reaches 75% of the full label gain in 300 questions where random needs 600, and ends 0.0009 above it. Optimising the model and optimising the decision are different objectives, and only one of them is what the system is for.

Classifier uncertainty is no better than random. It selects articles that are hard to classify, which correlates only loosely with where the expert helps. Expert uncertainty is worse than random, ending at 0.9152: chasing the points where the competence model is least sure gives it a badly skewed training sample, and its deferral rate collapses to 0.030, meaning it has talked itself out of using the expert at all.

6. Limitations

The expert is simulated, and simulated in a way the rejector can learn. Competence is piecewise constant over k-means regions of the same feature space the rejector uses, so a linear model on those features is well matched to the thing it has to fit. A real expert's competence would not be so conveniently shaped, and the AUC of 0.7498 should be read as an upper bound on what this approach would achieve against a human.

One expert, always available. There is no cost per query at deployment time, no queue, no fatigue, and no second expert to choose between. A budget on deferrals at test time, rather than only on training queries, would change the operating point.

The classifier is frozen during active learning. Expert answers are used only to learn competence, never to improve the classifier. In a real deployment an expert's answer is also a label, and using it for both is the obvious extension.

The training set is a sample. Using all 120,000 articles would raise the baseline by roughly a point and shrink the headroom deferral has to work with, but should not change which strategy wins.

7. Reproducing this report


```
python manage.py run_experiments # rewrites results/results.json  
python manage.py build_report # rewrites this PDF
```

Every step is seeded (data sample, split, k-means, expert answers, model fitting, query selection), so a rerun reproduces these numbers exactly. The run takes about 29.9 seconds. The interface at /project3/ reads the same JSON file, so the page and this report can never disagree.